CSCI160 Computer Science I

Lab: Functional Decomposition, Modular Programming, and Geometry Utilities

---

Objectives

- Separating logic from user interaction using multiple `.py` files
- Function definition and calling
- Function documentation using type hints
- Passing arguments into and returning values from functions
- Writing functions that call other functions (both built-in and user-defined)
- Input validation and formatting output
- Clear function naming for contextual use
- Documentation via comments and citations

Part 1: Task Summary

Build a menu-driven program that allows users to calculate the area of:

1. A circle
2. A rectangle
3. A triangle
4. Or exit the program

This project should use **three separate files**:

- `menu.py` – Contains functions to display the menu and handle menu input validation
- `shapes.py` – Contains functions to calculate shape areas
- `application.py` – The main program that brings everything together

Key Requirements

- Display a numbered menu of options
- Loop until the user selects "Exit"
- Prompt the user for shape-specific dimensions (e.g., radius, base, height)
- Call the appropriate function from `shapes.py` and display the result
- Use meaningful output formatting (e.g., rounding or labeling)

File Structure

```
shape_calculator/
├── application.py   # main loop and input/output
├── menu.py          # menu display and input validation
└── shapes.py        # shape area functions
```

Note: Have VS Code open the **`shape_calculator`** folder!!!

Task One: In `menu.py` define a function called **`show_menu`** that accepts a filename as an argument and displays a the file contents as a menu.

- The function signature should be: **`def show_menu( file_name: str )-> int`**
- This signature states that the function requires a string as an argument and returns an integer
- The menu is contained in a text file called `menu.txt` and contains three items
  - Circle Area
  - Rectangle Area
  - Triangle Area
- This function should open the file and format and print the contents as a menu and return the number of items in the menu. Given these three items, the menu should appear like this and the function would return 4

```
1) Circle Area
2) Rectangle Area
3) Triangle Area
4) Exit
```

- **Milestone, check your work:** Inside the `menu.py` file you can test this function by including a call.

```
if __name__ == "__main__":                  # is the script being run directly?
    n = show_menu("menu.txt")               # call the function and get the number of items
    print(f"The menu has {n} items.")       # print the number of items
```

This should display

```
1) Circle Area
2) Rectangle Area
3) Triangle Area
4) Exit
The menu has 4 items.
```

Task Two: Write the function called **`get_choice( )`** that accepts an integer representing the number of choices and performs the following task:

The function should prompt the user to enter their choice from. Given the example menu from above, the choices are [1, 2, 3, 4].

The choice will then be returned from the function. It is the number of the choice that should be returned.

This function should also validate the choices as they are entered. If the user chooses anything other than {1 - 4} that is an invalid choice. Display an error message and allow the user to choose again. This should happen indefinitely (while loop)

- The function signature should be **`def get_choice( n: int )-> int`**
- Validate the user choices for numeric values from 1 – n. Allow them infinite choices
- **Milestone, check your work:** Add the function call to your test area to ensure that it works correctly

```
if __name__ == "__main__":                  # is the script being run directly?
    n = show_menu("menu.txt")               # call the function and get the number of items
    print(f"The menu has {n} items.")       # print the number of items
    print()                                 # blank line
    choice = get_choice(n)                  # call the function to get the user's choice
    print(f"\nYou chose {choice}.")          # print the user's choice
```

The above code with the user choices of `[0, “a”, 5, 3]` should display



```
1) Circle Area
2) Rectangle Area
3) Triangle Area
4) Exit
The menu has 4 items.

Enter your choice: 0
Please enter a number between 1 and 4.
Enter your choice: a
Invalid input. Please enter a number.
Enter your choice: 5
Please enter a number between 1 and 4.
Enter your choice: 3

You chose 3.
```

Task Three: Now you will move your test code from the bottom of `menu.py` to a new file called `application.py.`

- In the same folder, create a new Python file called: `application.py`
- Call the menu and choice functions from `menu.py.` In order to do this you must first import the module into `menu.py`
- There are 2 ways to do this
  - Import just the module name (no file extension!): `import menu`
  - If you do this you'll have to prefix each function call with the module name
  - Import the function names individually from the module:
    - `from menu import show_menu, get_choice` ←this is my preference

```
application.py > ...
1  from menu import show_menu, get_choice # import the functions from menu.py
2
3  n = show_menu("menu.txt")              # call the function and get the number of items
4  print(f"The menu has {n} items.")      # print the number of items
5  print()                                # blank line
6  choice = get_choice(n)                 # call the function to get the user's choice
7  print(f"\nYou chose {choice}.")        # print the user's choice
```

  - **Milestone, check your work:** run `application.py` and call the functions from `menu.py.` You should see identical performance from the earlier test.
  - **Learning Advice:** put a breakpoint on the line that calls the show_menu function and ***step into*** this program using the debugger. Watch how it jumps from file to file and back.

Task Four: Create the `shapes.py` module to contain various function definitions for basic shape areas.

- In this module you will define 3 functions. Their signatures are below. The type hints are required.
  - `rectangle_area(width: float, height: float) → float`
  - `circle_area(radius: float) → float`
  - `triangle_area(base: float, height: float) → float`
  - These functions should contain the formulas implied by their names and the values should be returned
  - **NOTE:** These functions should only perform the tasks that their name implies. Do not have any user input or output statements **inside of these functions**. That is not part of their task. These functions need to **accept the data** they will operate on and **return** the result of the operation. User Input and Output are separate tasks and should be defined elsewhere. Failure to do this will result in a loss of points.

- **Milestone, check your work:** To ensure that the functions are working properly include some test cases at the bottom of `shapes.py`
- Here are a couple of examples. This code is inside `shapes.py.` Notice how the output statements are outside of the function definitions and the function just deals with numbers and math

```
if __name__ == "__main__":
    # Test the functions
    c = circle_area(5)
    print("Circle area with radius 5:", c)
    r = rectangle_area(4, 6)
    print("Rectangle area with length 4 and width 6:", r)
    t = triangle_area(3, 4)
    print("Triangle area with base 3 and height 4:", t)

    # You can also run the functions directly
    print("Circle area with radius 5:", circle_area(5))
    print("Rectangle area with length 4 and width 6:", rectangle_area(4, 6))
    print("Triangle area with base 3 and height 4:", triangle_area(3, 4))
```

- Running `shapes.py` gives me

```
❯ python shapes.py
Circle area with radius 5: 78.53981633974483
Rectangle area with length 4 and width 6: 24
Triangle area with base 3 and height 4: 6.0
Circle area with radius 5: 78.53981633974483
Rectangle area with length 4 and width 6: 24
Triangle area with base 3 and height 4: 6.0
```

Task Five: Now that you have 3 files that are all working together for a single purpose you can put them together into a menu driven application.

- Inside of `application.py` write to code to combine all behaviors of `menu.py, shapes.py and application.py` by showing the menu of options and allowing the user to choose which shape's area they'd like to see. Then depending on the shape, prompt for appropriate measurements, call the appropriate function and display the results. **This should happen inside of application.py only!!**
- Don't worry about validating the shape measurements unless you want to for practice
- Here are a few examples. Understand that this should happen infinitely

```
❯ python application.py

1) Circle Area
2) Rectangle Area
3) Triangle Area
4) Exit
Enter your choice: 1
You chose circle. Enter a floating point radius for the circle area
123.45
The area of a circle with radius of 123.45 is 47877.57

Enter your choice: 2
You chose rectangle. Enter the length and height of the rectangle
123.45
98
The area of a rectangle with length of 123.45 and height of 98.0 is 12098.1

Enter your choice: 3
You chose triangle. Enter the base and height of the triangle
786
823
The area of a triangle with base of 786.0 and height of 823.0 is 323439.0

Enter your choice: 4
You chose to Exit . . . Buh Bye
```

```
❯ python application.py

1) Circle Area
2) Rectangle Area
3) Triangle Area
4) Exit
Enter your choice: 0
Please enter a number between 1 and 4.
Enter your choice: 5
Please enter a number between 1 and 4.
Enter your choice: -1
Please enter a number between 1 and 4.
Enter your choice: three
Invalid input. Please enter a number.
Enter your choice: 1
You chose circle. Enter a floating point radius for the circle area
654
The area of a circle with radius of 654.0 is 1343709.44

Enter your choice: 4
You chose to Exit . . . Buh Bye
```

Part 2: Task Summary

This task will allow you to experiment with functions that call other functions.

- Create a new Python file named `functions.py`
- Begin by defining the function:
  `def distance(x1: float, y1: float, x2: float, y2: float) → float`
- This function accepts two pairs of Cartesian coordinates of the form: (x1, y1) and (x2, y2) that represent two points. The function then calculates and returns the distance between the two points using the `distance formula`

The Distance Formula: Given the two points (x_1, y_1) and (x_2, y_2), the distance d between these points is given by the formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- To really drive the point of this lesson home, be sure to have your distance function import and call the `sqrt( )` function from Python's Math module. Writing a function that calls a Python function as part of its definition is very similar to writing a function that calls another function that you have also written.
- **Test Case:** `distance(1, 2, 3, 4) => 2.8284 (rounded to 4 digits)`

Let's now create an alias for the distance function to apply it to different contexts. This type of design will allow the code to be more readable in the desired context.

Context One: The length of a line

- Define the function
- `def line_length(x1: float, y1: float, x2: float, y2: float) → float`
- This function also accepts two pair of Cartesian coordinates but instead it returns the length of the line formed by the coordinate pairs (x1, y1) and (x2, y2)
- Instead of **redefining the distance formula** let's just have the l`ine_length( )` function call the `distance( )` function. This alleviates the need of having to repeat the math for the distance. It also gives us a nice, descriptive function name that pertains to determining the length of of a line.
- Accomplish this by **including a call to the distance function inside of the definition of the line_length function**. Simply pass the coordinates along.

```
def line_length(x1: float, y1: float, x2: float, y2: float) → float:
    return distance(x1, y1, x2, y2)
```

- **Test Case:** `line_length(line) => 2.8284 (rounded to 4 digits)`

Context Two: The radius of a circle

- Define the function
- `def radius(x1: float, y1: float, x2: float, y2: float) → float`
- Define this function exactly as the `line_length` function
- **Test Case:** `radius(circle_points) => 2.8284 (rounded to 4 digits)`

Prove that the functions all work. Screen shot below of the test area in my `functions.py` file below. No need to add this to your application file. It is merely a short exercise on functions calling other functions.

```python
if __name__ == "__main__":
    # Test the functions
    ex1     = 1
    why1    = 2
    ex2     = 3
    why2    = 4

    d = round( distance(ex1, why1, ex2, why2), 4)   # pass the ereturn from distance into the round function
    r = round( radius(ex1, why1, ex2, why2), 4)     # pass the return from radius into the round function
    l = round( line_length(ex1, why1, ex2, why2), 4)# pass the return from line_length into the round function

    print("Distance:", d)
    print("Radius:", r)
    print("Line Length:", l)
    print()
    # pass the return from distance into the round function
    # and pass the return from round into the print function -> can be infinite levels of nesting
    print("Distance:", round(distance(ex1, why1, ex2, why2), 4))
    print("Radius:", round(radius(ex1, why1, ex2, why2), 4))
    print("Line Length:", round(line_length(ex1, why1, ex2, why2), 4))
```

Submission:

- `functions.py, application.py, shapes.py, menu.py`

RUBRIC

1. Program Functionality (12 points)

Criteria	Points
Menu displays correctly from `menu.txt`, and choice validation works	2
Loop allows repeated input until "Exit" is selected	2
User input handled properly for each shape	2
Area calculated correctly for each shape using functions	2
Geometry functions (`rectangle_area`, `circle_area`, `triangle_area`) return correct values	2
Distance-related functions (`distance`, `line_length`, `radius`) use proper decomposition and return accurate results	2

---

2. Code Structure and Modularity (11 points)

Criteria	Points
`application.py`, `menu.py`, `shapes.py`, and `functions.py` used correctly and consistently	2
No input/output in `shapes.py` or `functions.py` — functions do *only* computation	3
`menu.py` reads from file and returns item count accurately	2
Proper use of imports to connect modules (e.g., `from menu import ...`)	2
Geometry utility functions call `distance()` correctly (no formula duplication)	2

---

3. Documentation and Testing (7 points)

Criteria	Points
Function signatures include proper type hints (`-> float`, etc.)	3
Code includes clear **inline comments** explaining logic	2
Test code provided in `shapes.py` and `functions.py` for all relevant functions	2

Automatic Deductions

Minus 20% (6 points) from total score if any of the following are true:

- Code is AI-generated and **not accompanied by detailed explanation and citation**
- Student submits code with **few or no comments explaining logic**
- Code lacks modularity (e.g., everything written in one file)

Bonus (Optional: +1 to +2 points)

Criteria	Points
Bonus for **additional input validation** (e.g., no negative dimensions)	+1
Bonus for extra formatting polish (e.g., ASCII menu styling, spacing)	+1
